Tap Tamak

Frontend & Mobile Technical Specification

Nuxt 3 · Vue 3 · Pinia · Capacitor (iOS/Android)

For use with Cursor IDE

1. Stack Decision & Rationale

Layer	Technology	Reason
Web App	Nuxt 3 + Vue 3	You already know Vue. SSR for customer SEO, SPA for cook dashboard.
Mobile	Capacitor 6	Wraps your Nuxt SPA → iOS + Android with zero React. Fastest solo-dev path.
State	Pinia	Official Vue 3 store. Setup Store syntax = full TypeScript + Composition API.
Styling	Tailwind CSS v4	Utility-first. No context switching. Works with Nuxt + Capacitor.
Maps	vue3-google-map	Vue 3 Composition API wrapper for Google Maps. Works in ClientOnly.
HTTP	Custom $fetch plugin	Nuxt built-in $fetch with JWT interceptor + auto-refresh on 401.
WebSocket	socket.io-client	Matches NestJS Socket.io backend. Wraps in composables.
Forms	VeeValidate + Zod	Schema-based validation matching backend DTOs.
Icons	Iconify (nuxt-icon)	Access to 200K+ icons. Zero bundle cost with on-demand loading.
Persistence	pinia-plugin-persistedstate	Persist auth (cookie) and cart (localStorage) across page loads.

Why NOT React Native + WebView (your initial idea)

WebView hybrid means maintaining two separate rendering environments (Nuxt web + RN shell), debugging cross-layer issues, and users perceive the non-native feel. Capacitor gives you the same result (web code in a native container) with far less setup and zero new framework to learn as a solo Vue developer.

When to reconsider Expo/React Native

Switch to Expo if: map animations feel sluggish on low-end Android devices, you need background geofencing at scale, or you hire React-experienced developers. But start with Capacitor for v1.

2. Project Structure (Nuxt Layers)

Nuxt Layers provide clean domain separation between customer and cook experiences while sharing auth, API infrastructure, and UI components — all in one deployable Nuxt app.

```
tap-tamak-web/
├── nuxt.config.ts            # Extends all layers, configures Capacitor build
├── app.vue
│
├── layers/
│   ├── base/                 # Shared across customer + cook
│   │   ├── components/
│   │   │   ├── ui/           # Design system: UiButton, UiCard, UiInput, UiModal,
│   │   │   │                 #   UiBadge, UiSkeleton, UiAvatar, UiRating, UiSheet
│   │   ├── composables/
│   │   │   ├── useAuth.ts    # isAuthenticated, isCook, login, logout
│   │   │   ├── useAPI.ts     # Typed wrapper around $api with error handling
│   │   │   ├── useGeolocation.ts  # Browser + Capacitor unified geolocation
│   │   │   ├── useNotifications.ts # WebSocket notif queue + toast
│   │   │   ├── useImageUpload.ts   # File pick, preview, upload progress
│   │   ├── stores/
│   │   │   ├── auth.ts       # Persisted in cookie (SSR-safe)
│   │   │   ├── notifications.ts
│   │   ├── middleware/
│   │   │   ├── auth.global.ts # Redirect unauthenticated → /login
│   │   │   ├── role.ts       # Block non-cooks from /cook/* routes
│   │   ├── plugins/
│   │   │   ├── api.ts        # $fetch instance with auth headers + 401 refresh
│   │   ├── types/
│   │   │   └── index.ts      # Shared TypeScript types matching Prisma models
│   │
│   ├── customer/             # Customer-facing experience
│   │   ├── pages/
│   │   │   ├── index.vue     # Home: search, categories, cook map preview, recommendations
│   │   │   ├── cooks/
│   │   │   │   ├── index.vue  # Cook listing + full map view
│   │   │   │   └── [id].vue   # Cook profile page (bio, menu, reviews)
│   │   │   ├── dishes/[id].vue
│   │   │   ├── cart.vue
│   │   │   ├── checkout.vue
│   │   │   ├── payment/
│   │   │   │   ├── success.vue
│   │   │   │   └── failed.vue
│   │   │   ├── orders/
│   │   │   │   ├── index.vue  # Order history (Active / Completed / Cancelled tabs)
│   │   │   │   └── [id].vue   # Live order tracking with map
│   │   │   ├── favorites.vue  # Dishes + Cooks tabs
│   │   │   ├── profile/index.vue
│   │   ├── components/
│   │   │   ├── CookCard.vue
│   │   │   ├── DishCard.vue
│   │   │   ├── CartSummary.vue
│   │   │   ├── OrderStatusTracker.vue
│   │   │   ├── PaymentMethodSelector.vue
│   │   │   ├── maps/
│   │   │   │   ├── CookLocationsMap.vue     # All cooks on map with clusters
│   │   │   │   └── DeliveryTrackingMap.vue  # Courier location live update
│   │   ├── composables/
│   │   │   ├── useCart.ts
```

```
■   ■   ■   ■■■ useCookSearch.ts
■   ■   ■   ■■■ useOrderTracking.ts  # WebSocket order + location updates
■   ■   ■■■ stores/
■   ■   ■   ■■■ cart.ts         # Persisted in localStorage
■   ■   ■   ■■■ customerOrders.ts
■   ■   ■■■ layouts/customer.vue  # Header + mobile bottom nav + cart badge
■   ■
■   ■■■ cook/                 # Cook dashboard
■       ■■■ pages/cook/
■       ■   ■■■ dashboard.vue
■       ■   ■■■ menu/
■       ■   ■   ■■■ index.vue  # Dish grid with add/edit/hide/delete
■       ■   ■   ■■■ [id].vue   # Dish form (create + edit)
■       ■   ■■■ orders/
■       ■   ■   ■■■ index.vue  # Incoming order queue with status actions
■       ■   ■   ■■■ [id].vue   # Order detail
■       ■   ■■■ earnings.vue   # Income / History / Statistics tabs
■       ■   ■■■ profile.vue
■       ■   ■■■ verification.vue
■       ■■■ components/
■       ■   ■■■ OrderQueueCard.vue
■       ■   ■■■ DishForm.vue
■       ■   ■■■ EarningsChart.vue  # recharts or chart.js
■       ■   ■■■ VerificationUpload.vue
■       ■■■ composables/
■       ■   ■■■ useCookOrders.ts   # WebSocket for incoming orders
■       ■   ■■■ useMenuManagement.ts
■       ■   ■■■ useEarnings.ts
■       ■■■ stores/
■       ■   ■■■ cookOrders.ts
■       ■   ■■■ cookMenu.ts
■       ■■■ layouts/cook.vue   # Sidebar nav + top bar (desktop-first dashboard)
■
■■■ public/
```

3. Page Routing Reference

Route	Component	Auth	Description
/	customer/pages/index.vue	No	Home feed: search, categories, popular cooks, recommendations
/cooks	customer/pages/cooks/index.vue	No	Cook listing + full-screen map toggle
/cooks/:id	customer/pages/cooks/[id].vue	No	Cook profile: bio, schedule, menu, reviews
/dishes/:id	customer/pages/dishes/[id].vue	No	Dish detail: photos, description, ingredients, nutrition, add to cart
/cart	customer/pages/cart.vue	Yes	Cart review with item controls
/checkout	customer/pages/checkout.vue	Yes	Address form + promo code + payment method selector
/payment/success	customer/pages/payment/success.vue	Yes	Order confirmation + link to order tracking
/payment/failed	customer/pages/payment/failed.vue	Yes	Failed payment with retry option
/orders	customer/pages/orders/index.vue	Yes	Order history — Active / Completed / Cancelled tabs
/orders/:id	customer/pages/orders/[id].vue	Yes	Live tracking: 4-step progress + map + courier location
/favorites	customer/pages/favorites.vue	Yes	Saved dishes + saved cooks with filter chips
/profile	customer/pages/profile/index.vue	Yes	Account info, addresses, payment methods
/login	base/pages/login.vue	No	Email/phone + password login
/register	base/pages/register.vue	No	Registration form (role selection)

Route	Auth Required	Description
/cook/dashboard	COOK	Today's orders summary, quick actions, earnings widget
/cook/menu	COOK	Dish management grid with status toggles and quick actions
/cook/menu/:id	COOK	Dish create/edit form with image upload
/cook/orders	COOK	Incoming order queue with status advancement buttons
/cook/orders/:id	COOK	Order detail with items, customer address, deadline timer
/cook/earnings	COOK	Income overview / Transaction history / Statistics with charts
/cook/profile	COOK	Cook public profile editor + kitchen photos + reviews tab
/cook/verification	COOK	Verification document upload form (new cook onboarding)

Auth Middleware Logic

```
// layers/base/middleware/auth.global.ts
export default defineNuxtRouteMiddleware((to) => {
  const authStore = useAuthStore();
  const publicRoutes = ['/', '/login', '/register', '/cooks', '/dishes'];
  const isCookRoute = to.path.startsWith('/cook');

  if (!authStore.isAuthenticated && !publicRoutes.some(r => to.path.startsWith(r))) {
    return navigateTo('/login');
  }
```

```
  if (isCookRoute && authStore.user?.role !== 'COOK') {
    return navigateTo('/');
  }
});
```

4. Pinia Stores

auth.ts — persisted in cookie (SSR-safe)

```
// layers/base/stores/auth.ts
export const useAuthStore = defineStore('auth', () => {
  const user = ref<User | null>(null);
  const accessToken = ref<string | null>(null);
  const refreshToken = ref<string | null>(null);

  const isAuthenticated = computed(() => !!accessToken.value);
  const isCook = computed(() => user.value?.role === 'COOK');

  async function login(credentials: LoginDto) {
    const res = await $api('/auth/login', { method: 'POST', body: credentials });
    user.value = res.user;
    accessToken.value = res.accessToken;
    refreshToken.value = res.refreshToken;
  }

  async function refreshAccessToken() {
    const res = await $fetch('/api/v1/auth/refresh', {
      method: 'POST', body: { refreshToken: refreshToken.value }
    });
    accessToken.value = res.accessToken;
    refreshToken.value = res.refreshToken;
  }

  function logout() { user.value = null; accessToken.value = null; refreshToken.value = null; }

  return { user, accessToken, refreshToken, isAuthenticated, isCook, login, logout, refreshAccessToken };
}, {
  persist: { storage: piniaPluginPersistedstate.cookies() }  // SSR-safe
});
```

cart.ts — persisted in localStorage

```
// layers/customer/stores/cart.ts
export const useCartStore = defineStore('cart', () => {
  const items = ref<CartItem[]>([]);
  const cookId = ref<string | null>(null);  // enforce single-cook cart

  const totalItems = computed(() => items.value.reduce((s, i) => s + i.quantity, 0));
  const subtotal = computed(() => items.value.reduce((s, i) => s + i.dish.price * i.quantity, 0));

  function addItem(dish: Dish, quantity = 1) {
    if (cookId.value && cookId.value !== dish.cookId) {
      throw new Error('DIFFERENT_COOK');  // Handle on frontend: show confirm dialog
    }
    cookId.value = dish.cookId;
    const existing = items.value.find(i => i.dish.id === dish.id);
    if (existing) existing.quantity += quantity;
    else items.value.push({ dish, quantity });
    syncWithServer();  // Debounced POST to /cart/items
  }

  function clearCart() { items.value = []; cookId.value = null; }
```

```
  return { items, cookId, totalItems, subtotal, addItem, clearCart };
}, {
  persist: { storage: localStorage }  // Client-only
});
```

cookOrders.ts — WebSocket-driven real-time orders

```
// layers/cook/stores/cookOrders.ts
export const useCookOrdersStore = defineStore('cookOrders', () => {
  const incoming = ref<Order[]>([]);
  const socket = ref<Socket | null>(null);

  function connectSocket() {
    socket.value = io('/api/orders', { auth: { token: useAuthStore().accessToken } });
    socket.value.on('new_order', (order: Order) => {
      incoming.value.unshift(order);
      usePushSound();  // play notification sound
    });
    socket.value.on('order_updated', (updated: Order) => {
      const idx = incoming.value.findIndex(o => o.id === updated.id);
      if (idx > -1) incoming.value[idx] = updated;
    });
  }

  async function advanceStatus(orderId: string, status: OrderStatus) {
    await $api(`/orders/${orderId}/status`, { method: 'PATCH', body: { status } });
  }

  onUnmounted(() => socket.value?.disconnect());

  return { incoming, connectSocket, advanceStatus };
});
```

5. API Layer Architecture

Custom $fetch Plugin with Auth Interceptor

```
// layers/base/plugins/api.ts
export default defineNuxtPlugin(() => {
  const config = useRuntimeConfig();
  const authStore = useAuthStore();

  const api = $fetch.create({
    baseURL: config.public.apiBaseUrl,  // http://localhost:3000/api/v1
    onRequest({ options }) {
      if (authStore.accessToken) {
        options.headers = {
          ...options.headers,
          Authorization: `Bearer ${authStore.accessToken}`,
        };
      }
    },
    async onResponseError({ response }) {
      if (response.status === 401 && authStore.refreshToken) {
        try {
          await authStore.refreshAccessToken();
        } catch {
          authStore.logout();
          await navigateTo('/login');
        }
      }
    },
  });

  return { provide: { api } };
});
```

Service Files (Repository Pattern)

Each domain has a service file with typed functions. Use these in Pinia actions and event handlers. Use useAsyncData in page-level setup.

```
// layers/customer/services/cook.service.ts
export const cookService = {
  list: (params: CookSearchParams) =>
    useNuxtApp().$api<PaginatedResponse<Cook>>('/cooks', { params }),

  nearby: (lat: number, lng: number, radius = 10) =>
    useNuxtApp().$api<Cook[]>('/cooks/nearby', { params: { lat, lng, radius } }),

  getById: (id: string) =>
    useNuxtApp().$api<CookWithMenu>(`/cooks/${id}`),
};

// In a page component (SSR-aware):
const { data: cook, pending } = await useAsyncData(
  `cook-${route.params.id}`,
  () => cookService.getById(route.params.id as string)
);
```

```
// In event handlers / Pinia actions (client-side):
await cookService.nearby(lat, lng, 5);
```

nuxt.config.ts — Key Configuration

```
// nuxt.config.ts
export default defineNuxtConfig({
  extends: ['./layers/base', './layers/customer', './layers/cook'],

  modules: ['@pinia/nuxt', '@nuxtjs/tailwindcss', 'nuxt-icon', '@vueuse/nuxt',
            'pinia-plugin-persistedstate/nuxt'],

  runtimeConfig: {
    public: {
      apiBaseUrl: process.env.NUXT_PUBLIC_API_BASE_URL ?? 'http://localhost:3000/api/v1',
      googleMapsApiKey: process.env.NUXT_PUBLIC_GOOGLE_MAPS_KEY ?? '',
      wsUrl: process.env.NUXT_PUBLIC_WS_URL ?? 'http://localhost:3000',
    }
  },

  // CRITICAL for Capacitor mobile build:
  // ssr must be false when running nuxt generate for mobile
  ssr: process.env.NUXT_SSR !== 'false',

  nitro: {
    preset: process.env.NUXT_SSR === 'false' ? 'static' : 'node-server'
  }
});
```

6. Key Composables

useGeolocation — Unified Browser + Capacitor

```
// layers/base/composables/useGeolocation.ts
export function useGeolocation() {
  const lat = ref<number | null>(null);
  const lng = ref<number | null>(null);
  const error = ref<string | null>(null);

  async function getCurrentPosition() {
    // Use Capacitor Geolocation if available (mobile), else browser API
    if (window.Capacitor?.isNativePlatform()) {
      const { Geolocation } = await import('@capacitor/geolocation');
      const pos = await Geolocation.getCurrentPosition();
      lat.value = pos.coords.latitude;
      lng.value = pos.coords.longitude;
    } else {
      navigator.geolocation.getCurrentPosition(
        (pos) => { lat.value = pos.coords.latitude; lng.value = pos.coords.longitude; },
        (err) => { error.value = err.message; }
      );
    }
  }

  return { lat, lng, error, getCurrentPosition };
}
```

useOrderTracking — Live Delivery Updates

```
// layers/customer/composables/useOrderTracking.ts
export function useOrderTracking(orderId: string) {
  const status = ref<OrderStatus | null>(null);
  const courierLat = ref<number | null>(null);
  const courierLng = ref<number | null>(null);
  const socket = ref<Socket | null>(null);

  onMounted(() => {
    socket.value = io(useRuntimeConfig().public.wsUrl + '/orders', {
      auth: { token: useAuthStore().accessToken }
    });
    socket.value.emit('join_order', orderId);
    socket.value.on('status_changed', (payload) => { status.value = payload.status; });
    socket.value.on('location_update', (pos) => {
      courierLat.value = pos.lat;
      courierLng.value = pos.lng;
    });
  });

  onUnmounted(() => socket.value?.disconnect());

  return { status, courierLat, courierLng };
}
```

useImageUpload — Camera + Gallery + Upload

```ts
// layers/base/composables/useImageUpload.ts
export function useImageUpload() {
  const preview = ref<string | null>(null);
  const uploading = ref(false);
  const uploadedUrl = ref<string | null>(null);

  async function pickAndUpload(source: 'camera' | 'gallery' = 'gallery') {
    let file: File;
    if (window.Capacitor?.isNativePlatform()) {
      const { Camera, CameraSource } = await import('@capacitor/camera');
      const photo = await Camera.getPhoto({
        source: source === 'camera' ? CameraSource.Camera : CameraSource.Photos,
        resultType: 'base64',
      });
      // Convert base64 to File for upload
      file = base64ToFile(photo.base64String!, 'photo.jpg');
    } else {
      file = await pickFileFromInput('image/*');
    }
    preview.value = URL.createObjectURL(file);
    uploading.value = true;
    const formData = new FormData();
    formData.append('file', file);
    const res = await useNuxtApp().$api<{url: string}>('/upload/image', { method: 'POST', body: formData });
    uploadedUrl.value = res.url;
    uploading.value = false;
  }

  return { preview, uploading, uploadedUrl, pickAndUpload };
}
```

7. Capacitor Mobile Setup

Installation & Configuration

```
# Install Capacitor
npm install @capacitor/core @capacitor/cli
npm install @capacitor/ios @capacitor/android
npx cap init "Tap Tamak" "kz.taptamak.app" --web-dir=".output/public"

# Add platforms
npx cap add ios
npx cap add android

# Required plugins
npm install @capacitor/camera @capacitor/geolocation @capacitor/push-notifications
npm install @capacitor/app @capacitor/preferences @capacitor/google-maps
npm install @capacitor-community/background-geolocation

// capacitor.config.ts
import { CapacitorConfig } from '@capacitor/cli';
const config: CapacitorConfig = {
  appId: 'kz.taptamak.app',
  appName: 'Tap Tamak',
  webDir: '.output/public',
  plugins: {
    SplashScreen: { launchShowDuration: 2000, backgroundColor: '#FFF8F3' },
    Keyboard: { resize: 'body', resizeOnFullScreen: true },
    PushNotifications: { presentationOptions: ['badge', 'sound', 'alert'] },
    GoogleMaps: { androidApiKey: process.env.ANDROID_MAPS_KEY, iosApiKey: process.env.IOS_MAPS_KEY },
  },
};
export default config;
```

Build Scripts

```
// package.json scripts
{
  "scripts": {
    "dev":       "nuxt dev",
    "build:web": "nuxt build",
    "build:mobile": "NUXT_SSR=false nuxt generate && npx cap sync",
    "open:ios":  "npx cap open ios",
    "open:android": "npx cap open android",
    "sync":      "npx cap sync"
  }
}
```

Important: maps need special handling on Capacitor

The @capacitor/google-maps plugin uses a "hole-punch" technique — it renders a native map BELOW the WebView and cuts a transparent hole in the WebView to show it. Avoid placing web UI elements over the map area. Wrap all web maps in .

Push Notifications Setup

```
// Call on app startup after login — layers/base/plugins/push.client.ts
import { PushNotifications } from '@capacitor/push-notifications';
import { Capacitor } from '@capacitor/core';

export default defineNuxtPlugin(async () => {
  if (!Capacitor.isNativePlatform()) return;
  await PushNotifications.requestPermissions();
  await PushNotifications.register();

  PushNotifications.addListener('registration', async ({ value: fcmToken }) => {
    // Send FCM token to your backend
    await useNuxtApp().$api('/users/me/device-token', {
      method: 'POST', body: { token: fcmToken }
    });
  });

  PushNotifications.addListener('pushNotificationReceived', (notification) => {
    // App is open — show in-app toast
    useNotificationsStore().addToast(notification.title, notification.body);
  });

  PushNotifications.addListener('pushNotificationActionPerformed', (action) => {
    // User tapped notification — navigate to relevant screen
    const data = action.notification.data;
    if (data.orderId) navigateTo(`/orders/${data.orderId}`);
  });
});
```

8. Google Maps Integration

Use vue3-google-map (npm i vue3-google-map). Wrap all map components in — maps are browser-only.

Cook Locations Map (Customer)

```
<!-- layers/customer/components/maps/CookLocationsMap.vue -->
<template>
  <ClientOnly>
    <GoogleMap :api-key="config.googleMapsApiKey" style="height: 400px" :center="userLocation" :zoom="13">
      <MarkerCluster>
        <AdvancedMarker v-for="cook in nearbyCooks" :key="cook.id"
          :position="{ lat: cook.latitude, lng: cook.longitude }"
          @click="selectedCook = cook">
          <div class="cook-pin">
            <img :src="cook.avatarUrl" class="w-8 h-8 rounded-full border-2 border-orange-500" />
            <span>{{ cook.rating }}</span>
          </div>
        </AdvancedMarker>
      </MarkerCluster>
      <!-- Cook popup on pin click -->
      <InfoWindow v-if="selectedCook" :position="selectedCook" @closeclick="selectedCook = null">
        <CookMapPopup :cook="selectedCook" />
      </InfoWindow>
    </GoogleMap>
  </ClientOnly>
</template>
```

Delivery Tracking Map (Customer — Live)

```
<!-- layers/customer/components/maps/DeliveryTrackingMap.vue -->
<template>
  <ClientOnly>
    <GoogleMap :api-key="config.googleMapsApiKey" style="height: 300px" :center="courierPos" :zoom="14">
      <!-- Customer's delivery address pin -->
      <AdvancedMarker :position="deliveryAddress" title="Your address" />
      <!-- Animated courier marker — updates via WebSocket -->
      <AdvancedMarker :position="courierPos" title="Courier">
        <div class="courier-pin animate-bounce">■</div>
      </AdvancedMarker>
      <!-- Route polyline from cook to customer -->
      <Polyline :path="routePath" :options="{ strokeColor: '#F47B20', strokeWeight: 3 }" />
    </GoogleMap>
  </ClientOnly>
</template>

<script setup>
const { courierLat, courierLng } = useOrderTracking(props.orderId);
const courierPos = computed(() => ({ lat: courierLat.value, lng: courierLng.value }));
</script>
```

9. Design System & UI Components

Token	Value	Usage
primary	#F47B20	Buttons, active states, highlights, badges
primary-light	#FFF8F3	Card backgrounds, hover states, page background
dark	#1A1A1A	Body text, headings
gray	#666666	Secondary text, placeholders, captions
border	#E8E8E8	Card borders, dividers, input borders
success	#22C55E	Delivered status, success toasts
warning	#F59E0B	Preparing status, low stock warnings
error	#EF4444	Cancelled status, validation errors

Tailwind Config (tailwind.config.ts)

```
export default {
  theme: {
    extend: {
      colors: {
        primary: { DEFAULT: '#F47B20', light: '#FFF8F3', dark: '#D4611A' },
        dark: '#1A1A1A',
        muted: '#666666',
      },
      fontFamily: {
        sans: ['Inter', 'ui-sans-serif', 'system-ui'],
      },
      borderRadius: { xl: '12px', '2xl': '16px', '3xl': '24px' },
      boxShadow: {
        card: '0 2px 12px 0 rgba(0,0,0,0.06)',
        bottom: '0 -2px 12px 0 rgba(0,0,0,0.06)',
      }
    }
  }
}
```

Shared UI Component Checklist

Component	Props	Notes
UiButton	variant(primary/outline/ghost), size, loading, disabled	Orange primary, full-width option for mobile
UiCard	padding, shadow, hover	White bg, 12px radius, card shadow
UiInput	label, placeholder, error, icon	Orange focus ring, floating label
UiModal / UiSheet	title, show, onClose	Sheet = bottom drawer for mobile, Modal for desktop
UiSkeleton	width, height, rounded	Pulse animation for loading states
UiRating	value, max, readonly, size	Orange filled stars
UiAvatar	src, name, size	Shows initials if no image

UiBadge	variant(status colors), text	Pill shape for order status, cook tags
UiBottomBar	cartCount	Mobile bottom navigation — 4 tabs + cart badge

10. Screen-by-Screen Implementation Notes

Screen	API Calls	State	Notes
Home (index.vue)	GET /cooks/nearby, GET /dishes, GET /cooks/top (parallel)	useAsyncData on mount	Lazy-load cook map preview. Debounce search input 300ms.
Cook Profile [id].vue	GET /cooks/:id (incl. menu + reviews)	useAsyncData	Tab switch between About/Menu/Reviews is client-side only — no ex
Cart (cart.vue)	GET /cart on mount, PATCH/DELETE /cart/items/:id on change	cartStore (optimistic UI)	Sync cart to server debounced 500ms. Show "different cook" confirm
Checkout	GET /users/me/addresses, POST /orders, POST /payments/initiate	Local form state	After initiate → redirect to payment provider URL. On return → check
Orders [id].vue	GET /orders/:id, WebSocket join_order	useOrderTracking composable	Poll order status every 30s as fallback if WebSocket drops.
Cook Orders	GET /orders, WebSocket new_order	cookOrdersStore	Play audio on new_order event. Show deadline countdown timer per
Cook Menu	GET /cooks/me/dishes, PATCH/DELETE /dishes/:id	cookMenuStore	Optimistic UI: toggle availability instantly, revert on API error.
Earnings	GET /earnings, GET /earnings/transactions, GET /earnings/statistics	useAsyncData (statistics)	Lazy-load chart data only when Statistics tab is active.
Favorites	GET /favorites/dishes + /favorites/cooks	favoritesStore	Heart button on any DishCard/CookCard calls POST/DELETE immed

11. MVP Build Order

Phase	Deliverable
1 — Auth	Login, register, JWT handling, protected routes, role guard
2 — Browse	Home feed, cook list, cook profile page, dish detail (read-only, no cart)
3 — Cart & Order	Add to cart, checkout form, create order, order history page
4 — Payment	Halyk ePay redirect flow, payment success/failed pages, order status polling
5 — Cook Dashboard	Cook login, menu management (CRUD), incoming orders queue, status advancement
6 — Real-time	WebSocket order tracking map, live courier location, cook new-order notifications
7 — Polish	Favorites, reviews, push notifications (FCM), Capacitor mobile build + test on device
8 — Extras	Kaspi + Freedom Pay, cook statistics charts, cook verification flow, earnings payout